Tenbin Protocol Audit Report

Prepared by 0xleastwood

Version 1.0

Lead Auditors

0xleastwood

February 12, 2026

Contents

1 About Researchers **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**
- 4.0.1 Core Components . . . 2
- 4.0.2 Supporting Modules . . . 3
- 4.0.3 Cross-Chain Adapters . . . 3
- 4.0.4 Areas of Security Focus . . . 3

5 Audit Scope **4**

6 Executive Summary **5**
- 6.0.1 Key Findings . . . 5
- 6.0.2 Comments . . . 5

7 Findings **8**
- 7.1 Critical Risk . . . 8
  - 7.1.1 Inverted mapping in signer delegation allows for arbitrary order signing . . . 8
- 7.2 Medium Risk . . . 10
  - 7.2.1 Nonce usage allows for delegated signers to unknowingly replay orders . . . 10
  - 7.2.2 ERC4626 exposes a `mint` function that does not include required checks . . . 10
  - 7.2.3 Revenue calculations break for vaults with multiple depositors . . . 10
  - 7.2.4 Multi-collateral support introduces integration and de-peg risks . . . 11
- 7.3 Low Risk . . . 12
  - 7.3.1 `StakedAsset` deployments with unconfigured cooldown do not enforce blacklist on share redemption . . . 12
  - 7.3.2 Missing price staleness checks in `OracleAdapter` . . . 12
  - 7.3.3 Protocol allows for full-delegation to signer where signer dictates order recipient and not the payer . . . 12
  - 7.3.4 Order execution introduces potential economic attacks through the mint/redeem flow . . . 13
  - 7.3.5 Vault configurations are immutable and do not allow for the use of different vault implementations in the future . . . 14
  - 7.3.6 Morpho V2 integration breaks when vault fees are configured . . . 14
  - 7.3.7 LayerZero integration introduces potential foot-guns . . . 14
  - 7.3.8 Token burns always require a spendable allowance . . . 15
- 7.4 Informational . . . 16
  - 7.4.1 `ORDER_TYPEHASH` is not EIP-712 compliant . . . 16
  - 7.4.2 `cooldownLength` is vaguely defined . . . 16
  - 7.4.3 ERC4626 already exposes underlying asset decimals . . . 16
  - 7.4.4 Inconsistent versioning . . . 16
  - 7.4.5 Unused `BASIS_PRECISION` constant . . . 17
  - 7.4.6 Oracle delta tolerance behaviour is unnecessary . . . 17
  - 7.4.7 Order verification does not revert with custom error message on price deviance check . . . 17
  - 7.4.8 Orders without expiry are not supported . . . 17
  - 7.4.9 Approved spenders are not supported when initiating token unstaking . . . 18
  - 7.4.10 `ExactOut` operations are strictly not supported in the `SwapModule` design . . . 18
  - 7.4.11 Unnecessary unlimited approval on all added collateral vaults . . . 18
  - 7.4.12 Manager account is not explicitly named . . . 19
  - 7.4.13 `RevenueModule` cannot readily pull in pending revenue . . . 19
  - 7.4.14 Redemptions are not being fulfilled through any queue system . . . 19
  - 7.4.15 Revenue withdrawal will revert instead of setting an upper bound on the `amount` parameter . . . 20

1 About Researchers

0xleastwood is an independent security researcher specializing in Web3 and smart contract security. Contracted directly by the Tenbin Labs team for this engagement, he brings extensive experience identifying vulnerabilities and strengthening the security posture of decentralized protocols.

2 Disclaimer

The security researchers make every effort to identify vulnerabilities within the allotted time but hold no responsibility for the findings in this document. A security audit does not endorse the underlying business or product. This engagement was time-boxed and focused solely on the security aspects of the Solidity implementation.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Tenbin is a multi-asset issuance framework for liquid, yield-bearing tokenized FX and commodity assets. Each asset token is backed 1:1 by USD-stablecoin collateral, with price exposure created through dated futures on traditional exchanges (e.g. CME) rather than holding the underlying asset directly. The majority of on-chain stablecoin collateral is deployed into low-risk yield strategies, while a minority is off-ramped to institutional custody accounts to margin the hedge positions. Revenue from on-chain yield offsets futures financing costs; any surplus is distributed to staked asset holders, making Tenbin assets yield-bearing. The smart contracts reviewed in this engagement implement the on-chain components of this architecture.

4.0.1 Core Components

Controller

The primary entrypoint for minting and redeeming asset tokens. KYC-approved entities sign EIP-712/ERC-1271 orders specifying collateral and asset amounts; a hardware-security-module-controlled minter executes them atomically. On mint, stablecoin collateral is split between a `CustodianModule` (off-ramped to fund futures margin) and a `CollateralManager` (on-chain yield) according to a configurable ratio. A Chainlink-backed oracle adapter acts as a price backstop, reverting orders whose implied price deviates from the oracle beyond a configurable tolerance. Redemptions are served directly from the `CollateralManager`'s on-chain liquidity; if collateral is actively deployed in vaults, the curator must withdraw prior to order execution. Signers may delegate other accounts to sign orders on their behalf, and approved recipients are managed per-signer.

AssetToken

An immutable ERC-20 (with ERC-2612 permit) representing a tokenized real-world asset such as gold (tGLD) or a foreign currency. Minting is restricted to a single `minter` address (the `Controller`), and burning always consumes a spender allowance. The single-minter design means on Ethereum the `LayerZeroOFTAdapter` (lock/unlock) is used for cross-chain transfers, while spoke chains use a `MintBurnOFTAdapter` set as the minter.

CollateralManager (UUPS Upgradeable)

Holds the on-chain portion of stablecoin collateral backing outstanding asset tokens. Each supported collateral has an associated ERC-4626 vault (e.g. Morpho V2) for yield generation. Revenue accounting is separated from

protocol collateral via a `lastTotalAssets` checkpoint pattern. Provides curator-controlled `deposit`/`withdraw` into vaults, capped `rebalance` withdrawals to custodians for margin top-ups, and 1inch-routed `swap` operations between collateral types with per-pair slippage and cap constraints.

StakedAsset

An ERC-4626 vault that allows holders to stake asset tokens in exchange for a yield-bearing staking token. When the protocol earns surplus revenue (on-chain yield minus futures financing costs), new asset tokens are minted as rewards and vest linearly over a configurable period. Unstaking requires a cooldown during which assets are held in an `AssetSilo`. A minimum-shares invariant mitigates first-depositor donation attacks. A restricted-account registry enforces KYC/AML compliance by blocking sanctioned addresses from staking, unstaking, and transferring.

4.0.2 Supporting Modules

RevenueModule

Orchestrates the revenue lifecycle by pulling yield from the `CollateralManager` and routing it back to the manager to cover off-chain hedging costs, to a multisig for protocol treasury, or minting new asset tokens as staking rewards.

CustodianModule

Receives the custodian portion of mint collateral and distributes it to whitelisted custodian accounts (institutional brokerages with FCM licenses) via a keeper. Custodian whitelisting is token-agnostic: a custodian is either approved or not, regardless of which stablecoin is being sent.

SwapModule

Executes exact-input collateral swaps through the 1inch Aggregation Router V6, with parameter sanitization ensuring source/destination tokens, amounts, and receivers match the caller-supplied constraints. Exact-output operations are not supported by design.

OracleAdapter

Normalizes Chainlink aggregator answers to 18-decimal USD prices for the `Controller`'s delta-tolerance check.

MultiCall

Access-controlled batching contract that enables atomic bundling of curator operations with mint/redeem calls (e.g. `[withdraw(), redeem()]`).

4.0.3 Cross-Chain Adapters

LayerZeroOFTAdapter / LayerZeroMintBurnOFTAdapter / LayerZeroOVaultComposer

LayerZero V2 integrations for omnichain asset and staking-token transfers. The mint-burn adapter acts as the `minter` on spoke chains, while the vault composer enables cross-chain deposit/redeem operations.

4.0.4 Areas of Security Focus

The following areas were identified as requiring particular attention during this security engagement:

Area	Description
Order Verification	EIP-712/ERC-1271 signature validation, nonce management, signer–payer–recipient checks, and delegate trust chains must correctly prevent replay, impersonation, and unauthorized minting.
Collateral Accounting	Revenue must be cleanly separated from protocol collateral inside the `CollateralManager`; vault deposit/withdraw interactions and the `lastTotalAssets` checkpoint must not allow double-counting or loss of yield.

Oracle Backstop	Decimal normalization, price-delta calculation, and tolerance enforcement in the `Controller` must hold across all supported collateral decimals (6–18) and oracle feed configurations.
Swap Constraints	Slippage tolerance, swap caps, and parameter sanitization across `CollateralManager` → `SwapModule` → 1inch router must prevent value extraction through poorly priced or oversized swaps.
Staking & Vesting	Cooldown reset logic, vesting reward calculation, minimum-share enforcement, and silo accounting must resist manipulation (e.g. zero-amount cooldown resets, donation attacks, sandwich of reward transactions).
Access Control	Role separation across multiple multisigs and HSMs, upgradeability gates on the `CollateralManager` proxy, and the restricted-account registry must enforce intended trust boundaries.
Cross-Chain Bridging	LayerZero adapter configurations, mint/burn authority on spoke chains, and vault composer flows must not introduce supply inflation or stuck-token scenarios.

5 Audit Scope

The audit scope comprised a manual security review of the Tenbin smart contracts package and its subdirectories.

The following components were included in-scope:

Core Contracts

- `AssetSilo.sol`
- `AssetToken.sol`
- `CollateralManager.sol`
- `Controller.sol`
- `MultiCall.sol`
- `StakedAsset.sol`

Modules

- `CustodianModule.sol`
- `RevenueModule.sol`
- `SwapModule.sol`

Adapters

- `OracleAdapter.sol`
- `adapters/LayerZeroMintBurnOFTAdapter.sol`
- `adapters/LayerZeroOFTAdapter.sol`
- `adapters/LayerZeroOVaultComposer.sol`

Interfaces

- `interface/ICollateralManager.sol`
- `interface/IController.sol`
- `interface/IMintableBurnable.sol`

- `interface/IOracleAdapter.sol`
- `interface/IRestrictedRegistry.sol`
- `interface/IRevenueModule.sol`
- `interface/IStakedAsset.sol`
- `interface/ISwapModule.sol`

External Interfaces

- `external/1inch/IAggregationRouterV6.sol`
- `external/chainlink/AggregatorInterface.sol`

6 Executive Summary

Over the course of 12 days, 0xleastwood reviewed the Tenbin Protocol smart contracts provided by Tenbin Labs. In this period, a total of 28 issues were found.

The security assessment of the Tenbin smart contracts package, the on-chain component of Tenbin's liquid, yield-bearing RWA issuance framework, identified several issues requiring attention before production deployment.

6.0.1 Key Findings

Delegation & Access Control: The most severe finding involves an inverted mapping in the `Controller`'s `setDelegateStatus()` function, allowing any whitelisted signer to self-authorize as a delegate for other signers and execute arbitrary mint/redeem orders on their behalf. Nonce tracking was also scoped to the signer rather than the order payer, enabling delegated signers to unknowingly replay orders.

Staking Vault: The `StakedAsset` contract inherits ERC4626 but did not override the `mint()` entrypoint, bypassing the minimum-shares check and restricted-account enforcement. Cooldown and redemption paths also lacked blacklist enforcement when cooldown was unconfigured.

Collateral Management: Multi-collateral support introduces de-peg risk where redeemers can selectively exit into the most valuable stablecoin, as the protocol assumes 1:1 convertibility across all accepted stablecoins. The `CustodianModule` whitelisting is also token-agnostic, meaning funds can be sent to custodians that do not support a given stablecoin. Vault configurations were immutable with no migration path, and revenue accounting breaks under Morpho V2 fee configurations or multi-depositor scenarios.

Order Execution: The hybrid off-chain/on-chain order lifecycle, where a KYC'd signer submits a signed order to the backend which hedges and then executes on-chain, presents economic attack surface: order payers can front-run execution by revoking approvals, exploiting the time window between hedge placement and on-chain settlement to arbitrage price volatility.

6.0.2 Comments

The contract system demonstrates a well-structured separation of concerns across minting, collateral management, staking, and revenue modules. This reflects Tenbin's broader architecture where on-chain smart contracts handle issuance and collateral while off-chain systems manage futures hedging on traditional venues.

The remediation commit introduced several notable architectural changes:

- **StakedAsset upgraded to UUPS**: `StakedAsset` was converted from an immutable contract to a UUPS upgradeable proxy pattern, and the `MIN_SHARES` invariant was removed entirely.
- **Revenue accounting overhaul**: The `CollateralManager` now derives revenue from its own vault share balance via `convertToAssets()` rather than relying on `totalAssets()`, directly addressing the multi-depositor concern. Loss scenarios now subtract from pending revenue rather than silently ignoring them. A new `convertRevenue()` function allows the rebalancer to decline revenue to offset operational costs, and `withdrawRevenue()` is now restricted to the `RevenueModule` address rather than a separate `COLLECTOR_ROLE`.

- **Collateral vault lifecycle**: `removeCollateral()` and `redeemLegacyShares()` were added, backed by an `EnumerableSet` for collateral tracking. `updateController()` no longer requires the caller to enumerate collaterals.
- **Swap slippage model replaced**: The BPS-based `swapTolerance` was replaced with a `minSwapPrice` model (minimum token-out per token-in), and the `SwapModule` now validates `srcReceiver`, enforces a no-partial-fills flag, verifies the full input is consumed, and includes a `rescueToken()` function.
- **Oracle adapter replaced**: The generic `OracleAdapter` was removed and replaced with a `GoldOracleAdapter` that uses `latestRoundData()` with staleness, round-validity, and positive-price checks, directly addressing the oracle staleness informational finding.
- **AssetToken burn interface reworked**: `IMintableBurnable` was replaced with `IBurnMintERC20`, adding a `burn(uint256)` self-burn entrypoint and conditional allowance consumption, directly addressing the low-severity finding.
- **LayerZero adapters removed**: All three LayerZero adapter contracts were deleted from the in-scope source tree.
- **Nonce cancellation**: A new `invalidateNonce()` function was added to the `Controller`, allowing payers to proactively cancel pending orders on-chain.
- **Role consolidation**: `KEEPER_ROLE` and `MULTISIG_ROLE` in `RevenueModule` were merged into `REVENUE_KEEPER_ROLE`; `CustodianModule.send()` was renamed to `offramp()` with a dedicated `CUSTODIAN_KEEPER_ROLE`.

Summary

Project Name	Tenbin Protocol
Repository	monorepo
Commit	8bbb9b293406...
Final Repository	contracts
Fix Commit	d04a30df7e3c...
Audit Timeline	Nov 24th - Dec 9th, 2025
Methods	Manual Review

Issues Found

Critical Risk	1
High Risk	0
Medium Risk	4
Low Risk	8
Informational	15
Gas Optimizations	0
Total Issues	28

Summary of Findings

[C-1] Inverted mapping in signer delegation allows for arbitrary order signing	Resolved
[M-1] Nonce usage allows for delegated signers to unknowingly replay orders	Resolved
[M-2] ERC4626 exposes a `mint` function that does not include required checks	Resolved
[M-3] Revenue calculations break for vaults with multiple depositors	Resolved
[M-4] Multi-collateral support introduces integration and de-peg risks	Acknowledged
[L-1] `StakedAsset` deployments with unconfigured cooldown do not enforce blacklist on share redemption	Resolved
[L-2] Missing price staleness checks in `OracleAdapter`	Resolved
[L-3] Protocol allows for full-delegation to signer where signer dictates order recipient and not the payer	Acknowledged
[L-4] Order execution introduces potential economic attacks through the mint/redeem flow	Acknowledged
[L-5] Vault configurations are immutable and do not allow for the use of different vault implementations in the future	Resolved
[L-6] Morpho V2 integration breaks when vault fees are configured	Acknowledged
[L-7] LayerZero integration introduces potential foot-guns	Resolved
[L-8] Token burns always require a spendable allowance	Resolved
[I-01] `ORDER_TYPEHASH` is not EIP-712 compliant	Resolved
[I-02] `cooldownLength` is vaguely defined	Resolved
[I-03] ERC4626 already exposes underlying asset decimals	Resolved
[I-04] Inconsistent versioning	Resolved
[I-05] Unused `BASIS_PRECISION` constant	Resolved
[I-06] Oracle delta tolerance behaviour is unnecessary	Acknowledged
[I-07] Order verification does not revert with custom error message on price deviance check	Acknowledged
[I-08] Orders without expiry are not supported	Acknowledged
[I-09] Approved spenders are not supported when initiating token unstaking	Resolved
[I-10] `ExactOut` operations are strictly not supported in the `SwapModule` design	Acknowledged
[I-11] Unnecessary unlimited approval on all added collateral vaults	Acknowledged
[I-12] Manager account is not explicitly named	Acknowledged
[I-13] `RevenueModule` cannot readily pull in pending revenue	Acknowledged
[I-14] Redemptions are not being fulfilled through any queue system	Acknowledged
[I-15] Revenue withdrawal will revert instead of setting an upper bound on the `amount` parameter	Acknowledged

7 Findings

7.1 Critical Risk

7.1.1 Inverted mapping in signer delegation allows for arbitrary order signing

Context: Controller.sol#L168-L174, Controller.sol#L352

Description: The `Controller` contract facilitates delegated signing from the delegator (order payer) to the delegatee (permissioned signer). The mapping in `verifyOrder()` shows this indexed as `delegates[order.payer][signer]` whereas the `setDelegateStatus()` function shows it being set as `delegates[signer][msg.sender]`.

Because the mapping is inverted in this function, `msg.sender` can become a signer on any arbitrary `order.payer` as long as this account is also a whitelisted signer. In the case of the protocol, order payers are KYC'd and therefore whitelisted to sign their own orders if they desire. Delegatees do not have this requirement as it is assumed they have direct permission from the whitelisted order payer.

This would allow for specific abuse where a single signer can set malicious permissions for other whitelisted signers to execute mints/redeems on their behalf.

This can be showcased by adding the unit test to `Controller.t.sol`:

```
function test_VerifyOrder_Allows_UnauthorizedSelfDelegation() public {
    address attacker = recipient; // signer we control
    address orderRecipient = user;

    // whitelist payer (so it can be used as the signer argument) and attacker
    allowSigner(payer);
    allowSigner(attacker);

    // attacker allows their own recipient for future orders
    vm.prank(attacker);
    controller.setRecipientStatus(orderRecipient, true);

    // attacker signs an order on behalf of payer (no delegation yet)
    IController.Order memory order = IController.Order({
        order_type: IController.OrderType.Mint,
        nonce: 0,
        expiry: block.timestamp + 1000,
        payer: payer,
        recipient: orderRecipient,
        collateral_token: address(collateral),
        collateral_amount: 1_000e6,
        asset_amount: 1e18
    });
    IController.Signature memory signature = signOrder(recipientKey, hashOrder(order));

    // Without delegation this should revert
    vm.expectRevert(IController.InvalidPayer.selector);
    controller.verifyOrder(order, signature);

    // Attacker self-authorizes as delegate for payer without payer involvement
    vm.prank(attacker);
    controller.setDelegateStatus(payer, true); // writes delegates[payer][attacker] = true
    assertTrue(controller.delegates(payer, attacker));

    // Now verifyOrder succeeds using the attacker's signature on behalf of payer
    (address signer,) = controller.verifyOrder(order, signature);
    assertEq(signer, attacker);
}
```

Recommendation: Consider updating the `setDelegateStatus()` function to set the `delegates` mapping correctly. This would be implemented as follows:

```
/// @notice Allow an account to delegate a signer to sign orders on their behalf
/// @param signer Signer account to delegate to
function setDelegateStatus(address signer, bool status) external {
    if (!signers[signer]) revert InvalidSigner();
    delegates[msg.sender][signer] = status;
    emit DelegateStatusChanged(signer, msg.sender, status);
}
```

Tenbin Labs: Fixed in PR 397 by updating the `setDelegateStatus()` function to set the mapping the correct way.

0xleastwood: Verified fix.

7.2 Medium Risk

7.2.1 Nonce usage allows for delegated signers to unknowingly replay orders

Context: Controller.sol#L351, Controller.sol#L415-L421

Description: When an order is executed by the backend, the order is marked as used by consuming the nonce after order verification. However, if the order payer has delegated to multiple signers and various signers have passed on the same order to the backend, all of these orders will be executable because the `nonces` mapping is being indexed by the signer and not the order payer.

Consequently, there may be cases where multiple orders containing the same details are executed by the backend.

Recommendation: Consider changing how the `nonces` mapping is indexing order execution to use `nonces[order.payer][order.nonce]`.

Tenbin Labs: Fixed in PR 392 by making the `nonces` mapping track the order payer and not the signer.

0xleastwood: Verified fix.

7.2.2 ERC4626 exposes a `mint` function that does not include required checks

Context: StakedAsset.sol#L193-L203

Description: The `StakedAsset` contract distributes protocol yield to asset token holders who are willing to lock up their tokens via an ERC4626 style vault. Currently, the `ERC4626.deposit()` function is being overridden to include checks that would prevent users from manipulating the vault's exchange rate and enforce a blacklist as required for regulation compliance.

However, ERC4626 also exposes a `mint()` function which should be explicit if it is intended to be supported or revert otherwise. If not, it becomes possible for an attacker to bypass the `_checkMinShares()` logic and manipulate the exchange rate on vault deployment.

> Note: it is worth noting that `StakedAsset` deployments will be facilitated through a factory contract which will validate an initial mint.

Recommendation: Ensure the exposed `ERC4626.mint()` function is being overridden to include the `nonRestricted()` modifier and `_checkMinShares()` checks.

Additionally, it may be worthwhile transferring the minted shares from the factory contract to a dead address such that the `_checkMinShares()` check can be removed altogether. Effectively, the `DEFAULT_ADMIN_ROLE` can recover these funds by calling the `transferRestrictedAssets()` function on this same dead address.

Tenbin Labs: Fixed in PR 382 by adding the `nonRestricted()` modifier to the `mint()` function.

0xleastwood: Verified fix.

7.2.3 Revenue calculations break for vaults with multiple depositors

Context: CollateralManager.sol#L337-L357, CollateralManager.sol#L223-L263

Description: The protocol intends to use Morpho V2 vaults which allows for the vault owner/curator to gate specific parts of the vault system. It is unclear whether the collateral vaults added to the `CollateralManager` contract will be gated to ensure each vault is unique for each derivative asset supported by the protocol.

Ultimately, this breaks when other users are able to deposit into a Morpho V2 vault as the protocol will realize revenue whenever a new deposit is made. The allocation of assets by the curator of this parent Morpho vault is handled correctly and safely in V2 but there are inherent limits to the number of active and allocated vaults the protocol can support.

Recommendation: Ensure access gates are correctly configured for the deployed Morpho V2 vaults and that the vault curator is a guarded role. If it is intended that the `CollateralManager` contract supports multiple depositors, then realized revenue should be calculated by tracking the contract's invested share balance and readily redeem shares when the total share -> asset conversion exceeds the invested amount.

i.e. CAP also has an implementation of this here.

Tenbin Labs: Fixed in PR 468. An internal `_totalAssets()` function has been added to track vault balance based on share balance to prevent counting direct vault deposits as revenue.

0xleastwood: Verified fix.

7.2.4 Multi-collateral support introduces integration and de-peg risks

Context: Controller.sol#L355, Controller.sol#L282, CustodianModule.sol#L61-L69

Description: While initially, the protocol may only support a single collateral type for asset tokens mints and redemptions, it is set up to accept a variety of stablecoins for this. The oracle and backend make an assumption that a given stablecoin is convertible 1:1 to the $ amount of each asset token.

In the event of a de-peg, users have the option to redeem their asset token for any supported stablecoin and hence could opt to avoid realizing losses on a de-peg by redeeming the most valuable stablecoin backing the asset token. Ideally, redemptions should realize any existing de-peg losses by socializing losses across all users redeeming the asset token, regardless of which stablecoin is chosen to satisfy asset redemption.

It is also noted that whenever a mint is made, part of the order amount is sent to the `CustodianModule` contract which allows a designated keeper to send funds to whitelisted custodian accounts. Not all custodian accounts will support the same set of allowed stablecoins as per the `Controller` contract. Hence, there are issues with sending funds to a custodian account which does not integrate/support the target stablecoin which may or may not be capable of being remedied.

Recommendation: Consider implementing a mechanism to mint/redeem asset tokens while maintaining a target allocation of each collateral with a mint/redeem fee that is adjusted based on how order execution changes the backing of the asset token. While it is still expected that the asset token price will adjust to take on this loss at the point of a de-peg, any subsequent mints/redeems will be priced accordingly.

Alternatively, it may make more sense to support only a single collateral and abstract the management of these funds to the `CollateralManager` and `CustodianModule` contracts which can handle collateral swaps. A router contract can be implemented to handle swapping to the supported collateral on order signing and subsequent execution such that several stablecoins may be supported for asset mints.

Further validation should also be added in the `CustodianModule` contract to index supported custodians by which tokens it can readily accept. i.e. `if (!custodians[account][token]) revert NotApprovedCustodian();` would be safer to check in `CustodianModule.send()`.

Tenbin Labs: In the case that a collateral de-pegs, it's likely we do any of the following:

- pause mint & redemptions
- remove the collateral as an asset
- censor orders via backend that use a de-pegged asset (we check the collateral price via backend)

0xleastwood: Market makers are likely to take advantage of any de-peg situation before the protocol can safely handle this because in each case there is a path for quick value extraction:

- **pause mint & redemptions:** asset token sell pressure effectively marks the de-peg loss.
- **remove the collateral as an asset:** there is a run to quickly redeem any collateral that holds more value against the de-pegged stablecoin.
- **censor orders via backend that use a de-pegged asset:** this is effectively the same as removing the collateral as an asset and has the same consequences.

Tenbin Labs: Acknowledged.

0xleastwood: Acknowledged.

7.3 Low Risk

7.3.1 `StakedAsset` deployments with unconfigured cooldown do not enforce blacklist on share redemption

Context: StakedAsset.sol#L211-L221

Description: The `StakedAsset` contract has an optional cooldown on withdrawals/redemptions if configured. When this is not configured, the contract facilitates share redemptions on a first come first serve basis. However, in this case, it should also enforce the blacklist through the use of the `nonRestricted()` modifiers.

Recommendation: Ensure the blacklist is enforced on all functions that are a part of the typical user flow.

Tenbin Labs: Fixed in PR 384 by adding `nonRestricted()` checks to both affected functions.

0xleastwood: Verified fix.

7.3.2 Missing price staleness checks in `OracleAdapter`

Context: OracleAdapter.sol#L21-L25

Description: The `OracleAdapter` contract is used as a backstop to prevent the backend from executing badly priced orders. However, it is currently missing some simple price staleness checks to ensure the round is complete and fresh.

Recommendation: Consider adding these checks to ensure orders are performing price deviance checks with the most up-to-date oracle data.

Tenbin Labs: Fixed in PR 456.

0xleastwood: Verified fix. The `OracleAdapter` contract has been refactored to `GoldOracleAdapter` and includes price freshness checks.

7.3.3 Protocol allows for full-delegation to signer where signer dictates order recipient and not the payer

Context: Controller.sol#L347, Controller.sol#L158-L166

Description: Whenever a mint/redeem order is executed by the backend on-chain, the order is being verified before invalidating the nonce to prevent re-use and ensure order details are permitted. When an order payer delegates to a signer, this delegated account effectively has full permission to sign orders on behalf of the payer including the recipient of the minted or redeemed asset token.

There is a caveat in this behaviour, where if `setRecipientStatus()` is called/checked by the order payer and not the signer, it would allow full-delegation while ensuring that the minted/redeemed amount was sent to the payer's target recipient address. Effectively, the order payer is whitelisting recipients and not the delegated signer.

Recommendation: Confirm whether this is intended behaviour and consider making the order payer whitelist recipients instead of the signer.

Tenbin Labs: Acknowledged. This is the correct process. The signer should decide who is a recipient.

0xleastwood: Acknowledged. The order payer does full-delegation to the signer when calling `setDelegateStatus()`.

7.3.4 Order execution introduces potential economic attacks through the mint/redeem flow

Context: Controller.sol

Description: The typical mint/redeem flow for users is to first sign an order, send this to the protocol backend for execution which performs necessary checks for on-chain execution (including an approval check) and then spends gas to execute the order. It is assumed that the backend has access to better pricing than the on-chain oracle. There are also cases where no oracle is configured and therefore the backend has complete control over price execution.

The distinct time period where the order has been verified by the backend and not yet executed on-chain must be hedged by the protocol to ensure the price at which the user mints/redeems asset tokens is effectively "locked-in". This is required because without pre-confirmations, there is no guarantee of transaction inclusion and therefore, considerations must be made around front-running order execution with the removal of the token approval or potential chain re-orgs (longest one being 7-blocks since the merge). There is an inherent cost that the protocol bears and hence there are degrees of economic attacks in play here.

Because of this weird edge case where an order payer can front-run order execution and remove their approval on the contract, the order is technically still pending until one of the following *two* actions takes place:

1. A new order is executed on-chain with the same or higher transaction nonce which displaces this pending transaction.
2. The order payer re-approves the `Controller` contract to facilitate order re-execution.

In the case where a mint/redeem has not happened for some time, the user can re-add their approval and execute the mint/redeem at a potentially better price than originally, allowing the user to arbitrage price volatility in periods of low activity. This can be handled by invalidating the nonce on-chain after order execution failure.

> There is another consideration in regards to the direct manipulation of the futures orderbook, in which the backend would be forced to hedge an incoming order at a bad price. However, this is unlikely as it requires significant capital and sophistication to execute and can be easily prevented by adding mint caps to the `Controller` contract based on orderbook liquidity depth. It's also likely that this will be arbitraged before any on-chain action could take place.

Alternatively, through the approval removal, the protocol is forced to open large hedge positions before an order has been executed and considered re-org safe. There is an ongoing cost here in maintaining a hedge position if the order is not eventually being executed on-chain. However, there is a caveat in having KYC'd signers in the protocol as it will be made known to the protocol who the abusers are and they can be rate-limited in the backend.

There are also potential gas-griefing attacks whereby the cost of signing an order is free, allowing the payer to remove their approval on the contract before the backend is able to execute the order on-chain. Here, the backend pays the cost on-behalf of users to execute an order on-chain and this would most likely far exceed the approval removal because of how orders are being verified. There are *two* paths to defend against this:

1. Work closely with block builders to ensure order execution is not front-run by the order payer.
2. Do an early check that the `Controller` contract has an approval of at least `order.collateral_amount`, overall costing less gas than the approval removal by the order payer.

Recommendation: Consider updating the mint/redeem flow to invalidate a nonce even when order execution has failed. This would mean removing all reverts and using `Enum` error codes to return silently when an order is unable to be verified on-chain. Additionally, try/catch logic must be used when transferring tokens from `order.payer` to prevent any intentional reverts.

In the backend, abusers of this logic are KYC'd and can therefore be rate-limited/ignored to prevent any potential economic attacks where the protocol is paying more than the abuser. This can be included as part of the protocol's TOS or handled on-chain by removing the malicious signer from the `signers` mapping.

Tenbin Labs: Acknowledged.

0xleastwood: Acknowledged.

7.3.5 Vault configurations are immutable and do not allow for the use of different vault implementations in the future

Context: CollateralManager.sol#L130-L149

Description: The `CollateralManager` contract manages collateral vault deposits and withdrawals which effectively allow the vault curator to dictate how those assets are allocated in the Morpho V2 implementation.

Once a vault is configured, there is no possibility in the future in removing the supported collateral and its underlying vault. Consequently, without flexibility to change the collateral's target vault, it becomes impossible to unwind funds from one vault and migrate these over to another vault.

> Need to also be wary that vault adds are not being front-run by manipulating exchange rate to siphon funds off of the `CollateralManager`. But this can be done by vetting vault integrations beforehand. Should probably have a checklist that matches more or less what is being done to defend against the same attack with the `StakedAsset` contract.

Recommendation: Consider implementing logic to facilitate collateral vault migrations.

Tenbin Labs: Fixed in PR 398.

0xleastwood: Verified fix. The ability to remove a collateral and its underlying vault has been added, as well as a function to redeem legacy shares once a vault has been removed.

7.3.6 Morpho V2 integration breaks when vault fees are configured

Context: CollateralManager.sol#L345-L357

Description: The protocol maintains a Morpho V2 integration to manage vault deposits and withdrawals. Revenue calculations are reliant on `vault.totalAssets()` for determining the yield accrual amount owed to the `RevenueModule` contract.

If the underlying Morpho V2 vault has any performance and/or management fee configured, vault shares are effectively diluted over time and the current implementation of `CollateralManager` will treat this amount as effective revenue owed to the `RevenueModule`. This is because the Morpho V2 vault computes `vault.totalAssets()` by iterating through each adapter and its respective allocation and the unallocated amount that is held by the parent vault.

Recommendation: Ensure no performance/management fee is ever configured on the supported Morpho V2 vaults.

Tenbin Labs: Acknowledged. This has been added to the list of assumptions.

0xleastwood: Acknowledged.

7.3.7 LayerZero integration introduces potential foot-guns

Context: AssetToken.sol#L51-L62

Description: The current implementation of the `AssetToken` contract only allows for a single minter, which is canonically the `Controller` contract as it supports asset token mints and redemptions. As per the tests and current setup, the LayerZero integration uses the `LayerZeroMintBurnOFTAdapter` to facilitate token bridging on Ethereum which would require the `MINTER_ROLE` to have the ability to mint/burn tokens on behalf of any account. In its current state, users would not be able to burn and bridge tokens to another chain.

> Also a potential foot-gun, but the `LayerZeroMintBurnOFTAdapter` does not support tokens with less than 6 decimals.

The main benefit of using the `LayerZeroMintBurnOFTAdapter` is that there is no need for approvals because token burns will spend an allowance in `AssetToken`, which sort of negates this benefit.

Recommendation: Consider modifying the `AssetToken` contract to support multiple minters if the `LayerZeroMintBurnOFTAdapter` is intended to be used in this way. Otherwise, continue removing LayerZero dependencies in favor of a simpler approach.

Tenbin Labs: Addressed in PR 497. All LayerZero dependencies have been removed from the repository.

0xleastwood: Verified fix.

7.3.8 Token burns always require a spendable allowance

Context: AssetToken.sol#L58-L62

Description: The `AssetToken` contract has a public burn function that always treats `_msgSender()` as the spender and therefore will always consume an allowance even if `_from == _msgSender()`. This can be optimized to skip this allowance consumption when a self-burn is performed.

```
function burn(address _from, uint256 _amount) external returns (bool success) {
    _spendAllowance(_from, _msgSender(), _amount);
    _burn(_from, _amount);
    return true;
}
```

Recommendation: Consider only calling the internal `_spendAllowance()` function when `_from != _msgSender()`.

Tenbin Labs: Fixed in PR 428.

0xleastwood: Verified fix. Implemented as suggested.

7.4 Informational

7.4.1 `ORDER_TYPEHASH` is not EIP-712 compliant

Context: Controller.sol#L73-L76

Description: The `ORDER_TYPEHASH` is not EIP-712 compliant and should be corrected.

Recommendation: Ensure this matches the EIP-712 standard by updating `ORDER_TYPEHASH` to `Order(uint8 order_type,uint256 nonce,uint256 expiry,address payer,address recipient,address collateral_token,uint256 collateral_amount,uint256 asset_amount)`.

Tenbin Labs: Fixed in PR 384.

0xleastwood: Verified fix.

7.4.2 `cooldownLength` is vaguely defined

Context: StakedAsset.sol#L58-L59

Description: The naming of the `cooldownLength` seems to represent an array length variable rather than a time period.

Recommendation: Consider renaming `cooldownLength` to `cooldownPeriod`.

Tenbin Labs: Fixed in PR 383.

0xleastwood: Verified fix.

7.4.3 ERC4626 already exposes underlying asset decimals

Context: StakedAsset.sol#L205-L209

Description: The defined decimals for new asset tokens will always be the same and can therefore be hardcoded, but it is unnecessary to override this function when the ERC4626 implementation already exposes `_underlyingDecimals` through its own `ERC4626.decimals()` function. The `_underlyingDecimals` is always configured in the constructor to be the asset token's decimals.

Recommendation: Consider calling the already exposed `ERC4626.decimals()` function instead of hardcoding decimals.

Tenbin Labs: Fixed in PR 384 by calling the already implemented `decimals()` function as defined by the ERC4626 specification.

0xleastwood: Verified fix.

7.4.4 Inconsistent versioning

Context: Controller.sol#L81-L82, Controller.sol#L140

Description: The EIP-712 standard expects a string constant to be used to keep track of contract versioning. The existing `VERSION` constant is not being used for this purpose.

Recommendation: Consider replacing "1" in the EIP-712 inheritance with the existing `VERSION` constant.

Tenbin Labs: Fixed in PR 384.

0xleastwood: Verified fix.

7.4.5 Unused `BASIS_PRECISION` constant

Context: CollateralManager.sol#L61-L62, CollateralManager.sol#L323

Description: There is an unused constant `BASIS_PRECISION` in the `CollateralManager` contract.

Recommendation: Consider replacing the single instance of `10_000` with `BASIS_PRECISION` in the `CollateralManager` contract.

Tenbin Labs: Fixed in PR 381.

0xleastwood: Verified fix.

7.4.6 Oracle delta tolerance behaviour is unnecessary

Context: Controller.sol#L371-L375

Description: Whenever the `MINTER_ROLE` mints new asset tokens, the supported Chainlink oracle will be used to perform a price check against the signed order price at execution. Since the backend has the most up-to-date pricing of the asset and will be used as the canonical price source over Chainlink, this check is unnecessary and could allow for users to arbitrage the allowed tolerance.

Recommendation: Since the backend has the most up-to-date pricing, overwriting the price conversion within this tolerance could allow for better on-chain order execution.

Tenbin Labs: Acknowledged. The price oracle is used as a backstop - the order pricing is decided by the backend.

0xleastwood: Acknowledged.

7.4.7 Order verification does not revert with custom error message on price deviance check

Context: Controller.sol#L374

Description: The price deviance check is implemented as a backstop to prevent the backend from executing stale/bad orders on-chain. There is *one* `mulDiv()` operation being performed where OpenZeppelin's `Math` library will return its own error which is not being explicitly defined by the protocol.

Recommendation: Consider adding a custom error to propagate this revert.

Tenbin Labs: Acknowledged. This is a non-issue for us.

0xleastwood: Acknowledged.

7.4.8 Orders without expiry are not supported

Context: Controller.sol#L358

Description: When an order is signed and passed to the backend, the signer designates an expiration at which the order is no longer executable on-chain. This signer cannot set an order that does not expire by having `order.expiry == 0` and later giving an option to specifically invalidate the nonce on-chain which would effectively cancel the order.

Recommendation: Consider implementing persistent order signing or acknowledge this as a specific restriction. It may still be worthwhile to allow order signers to invalidate the order on-chain.

Tenbin Labs: Acknowledged. We will not support persistent orders using `order.expiry == 0`.

0xleastwood: Acknowledged.

7.4.9 Approved spenders are not supported when initiating token unstaking

Context: StakedAsset.sol#L110-L132

Description: The `StakedAsset` contract supports an unstaking cooldown when `cooldownLength` is configured. This effectively instantly withdraws the converted asset amount but does not release the tokens from the `AssetSilo` until the `cooldownLength` period has passed. Typical ERC4626 vault implementations allow for approved spenders to redeem/withdraw on behalf of a user, this being consistent with the `deposit()` and `redeem()` functions exposed when `cooldownLength` is not configured.

However, `cooldownShares()` and `cooldownAssets()` do not support this approved spending behaviour. Hence, the ability to allow the caller to be the spender on the tokens is lost, so the current implementation only partially supports ERC4626.

Recommendation: This can be handled by adding an `owner` argument to pass to the `_withdraw()` call which will use an allowance when `msg.sender != owner`. Hence, the recipient would still be `msg.sender` but would permit approved spenders initiating the unstake cooldown.

Tenbin Labs: Fixed in PR 499.

0xleastwood: Verified fix. Both the `cooldownShares()` and `cooldownAssets()` functions have been modified to allow for arbitrary `owner` parameters where an allowance is spent in the internal `ERC4626._withdraw()` function when `caller != owner`.

7.4.10 `ExactOut` operations are strictly not supported in the `SwapModule` design

Context: CollateralManager.sol#L299-L308, SwapModule.sol

Description: The `SwapModule` contract only supports a single integration (1inch) at the moment when facilitating swaps through the `CollateralManager` contract. It is assumed here that the entire input swap amount is being swapped when transferring `params.amount` to the `SwapModule` contract.

If multiple swap paths are to be supported in the future, this may lead to issues if the swap operation does not explicitly consume the whole input amount, leading to locked funds in the `SwapModule` contract. Additionally, this could break swap caps if they are updated with `params.amount` which may include an unconsumed amount.

Recommendation: Consider being explicit in the documentation that `ExactOut` operations are to never be supported by the `SwapModule` contract.

Tenbin Labs: Acknowledged.

0xleastwood: Acknowledged.

7.4.11 Unnecessary unlimited approval on all added collateral vaults

Context: CollateralManager.sol#L145

Description: When the `CollateralManager` contract adds a supported collateral, an unlimited approval is made to the underlying vault which continues to persist regardless of where assets are being allocated. Instead, this can be handled within the `CollateralManager.deposit()`.

Recommendation: Ensure there are as few lingering approvals in the protocol by moving vault approvals to the `CollateralManager.deposit()` function.

Tenbin Labs: Acknowledged.

0xleastwood: Acknowledged.

7.4.12 Manager account is not explicitly named

Context: Controller.sol#L210-L215

Description: The `Controller` contract designates a `manager` account as the recipient of a given ratio of collateral used in every mint. It is unclear who the intended recipient is here.

Recommendation: Clarify in the `setManager()` function comments or by re-naming the `manager` variable that this target recipient is the `CollateralManager` contract.

Tenbin Labs: Acknowledged. We are fine with this naming convention.

0xleastwood: Acknowledged.

7.4.13 `RevenueModule` cannot readily pull in pending revenue

Context: CollateralManager.sol#L224-L232, RevenueModule.sol#L76-L83

Description: All revenue accrued in the `CollateralManager` contract cannot be readily pulled into the `RevenueModule` contract because it may be actively deposited in one of the collateral's underlying vaults. This could lead to inconsistent yield accrual if the `CollateralManager` is actively deploying all capital.

Recommendation: Consider updating `CollateralManager.deposit()` to maintain a reserve `pendingRevenue[collateral]` when depositing that can always be pulled by the `RevenueModule` contract.

Tenbin Labs: Acknowledged. Typically we don't care about reserving revenue, since it is earning yield while deposited in vaults. The pattern we imagine is if there is no liquidity for revenue, shares can be redeemed by the `CURATOR_ROLE` before withdrawal is triggered by the `COLLECTOR_ROLE`.

0xleastwood: Acknowledged.

7.4.14 Redemptions are not being fulfilled through any queue system

Context: Controller.sol#L307, CollateralManager.sol

Description: The current flow for protocol deposits/redemptions is to distribute collateral to the `CollateralManager` and `CustodianModule` contracts on mint. The former manages vault deposits and readily redeemable collateral balances and the latter sends funds to whitelisted custodian accounts which manage funds on behalf of the protocol.

In the event of mass redemption, the protocol can only execute orders for where there are available funds in the `CollateralManager` contract to satisfy the request. If these funds are not readily available, funds must be withdrawn from the collateral's underlying vault or pulled from the custodian account directly. There are inherent time delays in this, which means any user can ensure their redemption request is satisfied by the backend first if they time their redemption upon new funds being made available.

Recommendation: If this is intended to be supported in the backend, ensure the queue system is enforced as a FIFO type queue system or consider implementing/enforcing this on-chain. Funds could also be readily withdrawn from the underlying vault when required to make this even more seamless and potentially remove the need to have the `CURATOR_ROLE` manage deposits and withdrawals.

Tenbin Labs: Acknowledged. This is the intended mechanism. If there is no liquidity in the collateral manager, redemption should not be possible.

0xleastwood: Acknowledged.

7.4.15 Revenue withdrawal will revert instead of setting an upper bound on the `amount` parameter

Context: CollateralManager.sol#L245-L263

Description: The `withdrawRevenue()` function does not allow the `COLLECTOR_ROLE` to execute a maximum withdrawal of revenue by setting `amount` to `type(uint256).max` and bounding this to `totalRevenue`.

Recommendation: Consider setting an upper bound on the `amount` parameter to be `totalRevenue` to support this behaviour.

Tenbin Labs: Acknowledged.

0xleastwood: Acknowledged.